Kinesis Dual Store Telemetry and Executor Isolation in a Real Time Pose Analytics Pipeline

Rishi Chalana
School of Computer Science and Engineering
Vellore Institute of Technology, Vellore, India
rishichalana2@gmail.com

Tanay Mehto
School of Computer Science and Engineering
Vellore Institute of Technology, Vellore, India
tanay.mehto2023@vitstudent.ac.in

Divyansh Agrawal
School of Computer Science and Engineering
Vellore Institute of Technology, Vellore, India
divyansh.agrawal2023a@vitstudent.ac.in

Dr. Adaline Suji R
School of Computer Science and Engineering
Vellore Institute of Technology, Vellore, India
adalinesuji.r@vit.ac.in

***Abstract*—Real time exercise analytics couples compute intensive pose inference with sustained telemetry writes. This paper presents Kinesis, a WebSocket based system that extracts 33 MediaPipe landmarks per frame, derives exercise metrics, and persists landmark trajectories in TimescaleDB with an optional Cassandra mirror. The storage schemas are shaped around a session and landmark trajectory query and use matched 90 day retention. A replicated load study compared TimescaleDB only, Cassandra only, and dual write configurations across eight concurrent sessions and 90 frames per session, with three runs before and after database flushes were moved from the four worker inference pool to a dedicated two worker pool. Executor isolation increased median per session throughput by 13 percent for both single store paths. Cassandra median flush latency fell from approximately 75 ms to 44 ms and write throughput rose from approximately 3.5 to 6.5 thousand rows per second. Dual write throughput remained about 13 to 14 frames per second because the sequential TimescaleDB and Cassandra flush was still awaited every tenth frame. All 18 runs had zero observed frame loss, and both stores contained exactly 2970 rows per session in dual write mode. The results separate worker pool interference from critical path latency and show that thread isolation improves resource independence but does not make an awaited mirror write free.**

***Index Terms*—human pose estimation, time series databases, Cassandra, TimescaleDB, real time systems**

I. Introduction

Camera based exercise systems must produce feedback quickly while retaining enough motion data for later analysis. A single tracked person generates 33 landmarks per frame. At 30 frames per second this is roughly 1000 coordinate rows per second for each session before rep summaries, form events, or user profiles are considered. The resulting pipeline is both a perception system and a time series ingestion system. Storage work can therefore interfere with the inference workers that sustain the live interaction.

MediaPipe provides a graph based framework for timestamped perception pipelines [1], while BlazePose supplies a single person topology with 33 keypoints and reported interactive performance for fitness oriented use cases [2]. Kinesis uses this pose front end on the server. The browser captures 640 by 480 webcam frames, sends JPEG payloads through an authenticated WebSocket, and receives landmarks, joint angles, rep state, asymmetry, tempo, compensation, and movement signature signals. Raw pixel frames are discarded after inference; only derived numeric data are stored.

The study asks two questions. First, how should the same high frequency landmark stream be represented in a relational time series store and a wide column store when the dominant read is one landmark trajectory within one session? Second, what changes when blocking database flushes stop sharing workers with pose inference? The system is relevant to fog and edge analytics because it studies bounded compute and persistence resources near a live data source, but the evaluated implementation performs inference on a backend server and is not presented as an edge deployment.

The contribution is an empirical systems analysis rather than a new pose model. It documents a query shaped dual store design, a controlled six condition benchmark with repeated runs, and a diagnosis that distinguishes cross session worker pool contention from latency that remains inside each session's request response path.

II. Related Work

A. Pose based exercise analysis

MediaPipe organizes perception components as calculators connected by timestamped packet streams and includes scheduling and flow control mechanisms for real time workloads [1]. BlazePose combines detection and tracking to estimate 33 body keypoints for one person [2]. A monocular MediaPipe study reported exercise dependent range of motion errors and identified occlusion and depth ambiguity as important limitations [3]. Kinesis builds on these components and focuses on the telemetry path created after inference. Its biomechanical rules use joint angles, calibrated movement ranges, and trends. The interpretation is bounded by established evidence that hip and trunk control affects knee mechanics [4] and that squat depth changes knee loading [5].

These sources motivate monitored signals; they do not validate Kinesis as a clinical device.

B. Time series and wide column storage

Time series benchmarks show that database performance depends on workload and access pattern. A comparison of relational, NoSQL, and specialized systems found different write and resource profiles [6], while a later parallel access study found that no tested system dominated every concurrency condition [7]. SciTS and TSM Bench likewise organize evaluations around representative ingestion and query workloads [8], [9]. Reproducible performance guidance calls for explicit experimental conditions and repeated measurements [10]. Kinesis therefore reports the workload, repetitions, percentiles, throughput, and correctness checks for one application path.

Cassandra data modeling is commonly organized around application queries [11]. The system itself was designed as a decentralized structured store whose partitioning and tunable consistency support scalable writes [12]. Kinesis does not attempt to prove general superiority over TimescaleDB. The Cassandra path is a best effort mirror evaluated on one development node, while TimescaleDB remains authoritative. The comparison exposes application level costs caused by different write APIs and by the placement of persistence work in the concurrency architecture.

C. Resource interference

Even the background IO processes may compete with the foreground computations in terms of CPU time, memory usage, and bandwidth [13]. In particular, the case is valid in case of asynchronous communication when the blocking tasks run under the veil of a worker pool. Kinesis experiment stands for the evaluation of a small but realistic intervention, where the database flushes, that were done earlier by the very same four workers used by MediaPipe, are now done by a new pool of two workers.

III. System Architecture

Figure 1 illustrates the data flow diagram. Frames are captured by the React client at the rate of 30 fps. It is then encoded into base64 jpeg and sent via the WebSocket connection with the authentication using the JSON Web Token. The FastAPI server validates the token before the connection is established. The frame is decoded in the memory and stored in the queue for MediaPipe to process the image using the FRAME EXECUTOR which is the pool of 4 threads used for all sessions. The Kinesis pipeline calculates the rep state, joint angles, bilateral asymmetry, tempo, compensation trends, and Dynamic Time Warping movement signature. Dynamic Time Warping makes it possible to compare trajectories even if the repetitions have different lengths [14].

Every frame receives 33 normalized coordinates that are then captured by the LandmarkBuffer and aggregated for 10 frames. When the connection is lost, the last batch of data is flushed. TimescaleDB is the first database where the data is written. In case the feature flag is enabled and Cassandra is running, the same data is written to Cassandra. In case the connection to Cassandra is lost, the data is dropped without performing the rollback of the authoritative write to avoid disruptions. This design allows us to maximize the availability of the live path but allows divergence to happen.

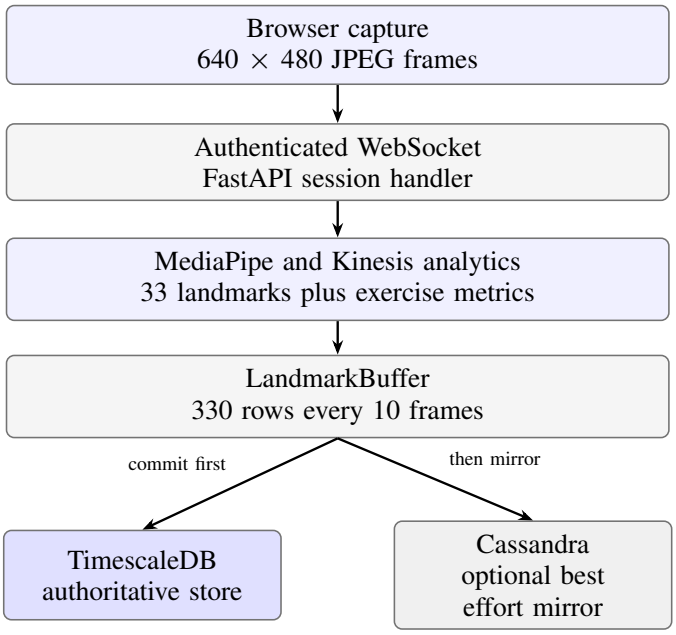


Fig. 1. Kinesis processing and dual store telemetry path

IV. Storage Design

A. TimescaleDB schema

The authoritative landmark_frames hypertable stores time, session identifier, frame identifier, landmark identifier, x and y normalized coordinates, and visibility. Its primary key contains time, session identifier, landmark identifier, and frame identifier. A secondary index on session identifier, landmark identifier, and descending time serves the hot trajectory read. Chunks older than seven days are compressed using session and landmark as segment keys and time as the ordering key. Raw landmarks expire after 90 days, while compact rep level summaries remain in ordinary PostgreSQL tables. Hypertables use time based chunks while retaining PostgreSQL indexes and transactions [15].

B. Cassandra schema

The Cassandra table uses the composite partition key session identifier and landmark identifier. Timestamp and frame identifier form the clustering columns, ordered by descending timestamp and ascending frame identifier. The access pattern therefore becomes a single partition scan whose physical order matches the trajectory query. A session only partition would be about 33 times wider and would mix landmarks that the query immediately separates. A day bucket was omitted because exercise sessions are short; longer or continuous deployments would require a bucket to bound partition growth.

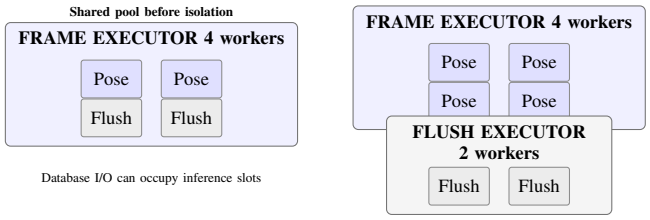


Fig. 2. Worker ownership before and after executor isolation

Every row is set with the table time to live of 90 days. From previous studies in Cassandra time series [16], it is important to analyze the relationship between time to live and the compaction strategy. In order to delete expired SSTables easily, TimeWindowCompactionStrategy allows for similar age immutable data to be grouped [17]. The configuration makes use of one day windows with a default 10-day gc grace period, since a smaller grace period will make it necessary to run repairs that have not yet been achieved in the project. Thirty-three rows in one frame indicate 33 partitions, thus prepared async inserts are used by the writer.

C. Consistency and write semantics

The replication factor is set to three in production, LOCAL ONE is write, and LOCAL QUORUM is read. One write replica plus two read replicas make up a total of three rather than more than three; therefore, this setup does not ensure read write intersection in the true meaning of quorum. This is a throughput-based decision, and telemetry is replaceable. Benchmarking in development was done using one Cassandra 5.0 instance with a replication factor of one. TimescaleDB is the single source of truth, and Cassandra is disabled by default.

V. Executor Isolation

The first implementation dispatched both pose inference and persistence flushes to FRAME EXECUTOR. A TimescaleDB flush holds a worker for roughly 20 ms under the benchmark load, while a Cassandra flush commonly holds one for about 75 ms before isolation. Concurrent sessions can therefore occupy inference slots with storage operations. The revised architecture introduces FLUSH EXECUTOR with two threads. Each session still awaits its own flush, preserving row order and preventing the buffer from being mutated concurrently, but storage no longer consumes the four inference workers.

This design isolates the two latencies. Contention of shared pool impacts other sessions as a flush consumes compute resources from the whole server. Latency on critical path impacts the flushing session irrespective of where the flush takes place. This test measures if the first latency is minimized while the second is observable.

VI. Experimental Method

Tests were performed using an Apple M2 development machine and Python 3.12, with the databases containerized. The application locks MediaPipe 0.10.14 and Cassandra 5.0. The TimescaleDB image is noted as latest pg18 and not as an immutable extension version, thus limiting replicability. Eight WebSocket clients concurrently sent 90 drawn figures' JPEG images which resulted in 33 landmarks. A client would only send the next frame once the response to the previous one had been received. Each condition had three iterations, before and after the isolation of the executor, for a total of 18 tests.

TABLE I
Benchmark Configurations

Config	TimescaleDB	Cassandra	Purpose
A	On	Off	Authoritative path
B	Skipped	On	Isolate mirror cost
C	On	On	Shipped dual write

TABLE II
Median Per Session Throughput

Path	Shared pool	Dedicated pools	Change
TimescaleDB	21.2 fps	23.9 fps	+13%
Cassandra	15.8 fps	17.8 fps	+13%
Dual write	14.3 fps	12.9 fps	Within noise

The main metric used for the application performance measurement was median frames per second per session. For the instrumented flushes, there were reports of median and 95th percentile latencies, in addition to rows per second per storage mechanism. A comparison of the row counts provided the correctness check, where each completed session should have 2970 rows (90 frames * 33 landmarks). The repeat measure design was used based on a prior single iteration which indicated a 15-18% persistence regression, but which is not included in the final causal assertion.

VII. Results

Executor isolation improved both single store paths. TimescaleDB only throughput increased from 21.2 to 23.9 frames per second, and Cassandra only throughput increased from 15.8 to 17.8 frames per second. Both changes are approximately 13 percent. The similar direction across independent stores supports the diagnosis that shared worker ownership, rather than one database's internal behavior alone, constrained live processing.

The clearest store level change occurred for Cassandra only. Its median flush latency fell from about 75 ms to 44 ms, and write throughput rose from about 3.5 to 6.5 thousand rows per second. TimescaleDB tail latency also narrowed. Cassandra still cost more per flush because TimescaleDB accepted one 330 row bulk operation while Cassandra created 330 asynchronous single partition inserts and waited for all futures.

Dual write throughput did not improve. The median changed from 14.3 to 12.9 frames per second, which the replicated measurements treated as noise rather than a benefit or regression. In this mode each tenth frame awaits a TimescaleDB commit followed by the Cassandra mirror, typically totaling 65 to 90

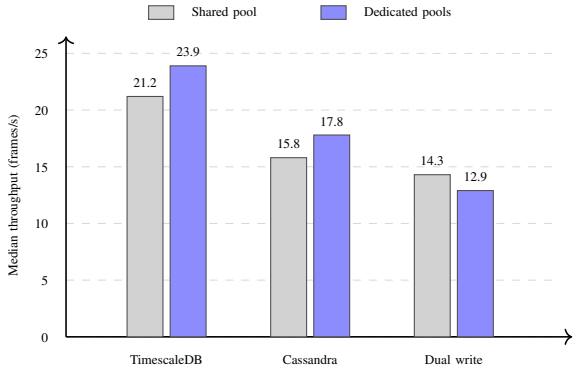


Fig. 3. Median per-session throughput before and after executor isolation. Values reproduce Table II; no additional measurements are introduced.

TABLE III
FLUSH LATENCY AND WRITE THROUGHPUT

Path and store	p50 before	p50 after	Rows per second before to after
A TimescaleDB	19 to 21 ms	18 to 20 ms	13.5 to 17k → 16 to 17.5k
B Cassandra	74 to 80 ms	43 to 53 ms	3.4 to 3.8k → 5.6 to 6.9k
C TimescaleDB	28 to 34 ms	23 ms	5 to 9k → 10.7 to 14k
C Cassandra	55 to 71 ms	41 to 45 ms	3.7 to 5k → 4.5 to 5.9k

ms. Moving that work does not remove it from the current session's response path. The result rejects the stronger claim that executor isolation solves dual write latency.

No frame loss was observed in any of the 18 runs. In dual write mode, TimescaleDB and Cassandra both stored exactly 2970 rows per session in every run. These checks establish lossless behavior for the tested self throttling harness. They do not establish lossless behavior for an unbounded producer or during a Cassandra outage.

VIII. DISCUSSION

Asynchronous placement is best understood from the perspective of the need for two different levels of analysis. A dedicated executor allows the inference capabilities of global processing to operate independently of storage requests while improving store isolation. The await at the session boundary is still a serial dependency and thus the dual path writes continue to suffer from mirror latency. Thread isolation is a resource ownership problem, but not an end-to-end latency problem.

A queue backed, fire-and-forget style mirror could yield after the TimescaleDB commit and reduce a critical path closer to the authoritative store's approximate 20 ms flush. That is not the design choice taken by Kinesis so far. A bounded queue will have to define its behavior in terms of blocking, dropping mirror telemetry or rejecting tasks in case of a lagging Cassandra. Failures need to be decoupled using queue depth, write failure and parity checks. Otherwise lower latency would hide divergence rather than prevent it.

The schema definitions show how an application-level comparison should start from the read query. TimescaleDB has a composite index and time-based chunks. Cassandra uses equality dimensions for its partition key and time for its clustering order. Equal retention periods for 90 days makes operationally meaningful intent comparable despite the different deletion mechanisms. The values obtained are implementation-specific and cannot be extrapolated to clustered Cassandra or other time-series workloads.

IX. LIMITATIONS

The baseline involved an Apple M2 server and one Cassandra node with replication factor one. The following aspects were not included in the performance evaluation: replica coordination, network partitioning, repair, storage space, and multiple node scalability. The tag for TimescaleDB Docker image was not fixed, and the test report did not contain all of the database tuning settings. Three iterations decrease the impact of an outlier but do not provide a formal statistical analysis.

The sequential send receive client applies natural backpressure. Zero observed frame loss means that every submitted frame received a response under this protocol; it does not predict behavior when a browser produces frames independently of response completion. Drawn figure images made landmark counts deterministic but did not test pose accuracy across bodies, lighting, occlusion, camera angles, or exercise technique. No clinical outcome study was performed. Accordingly, the paper evaluates systems performance and storage correctness, not biomechanical validity or injury prevention.

X. CONCLUSION

Kinesis demonstrates a reproducible systems lesson in a real time pose analytics workload. Query shaped TimescaleDB and Cassandra schemas can retain the same landmark trajectories, and a best effort mirror can be isolated from the authoritative write. Moving database flushes to a dedicated two worker executor increased both single store live paths by about 13 percent and nearly doubled Cassandra write throughput. The dual write path did not accelerate because the mirror remained awaited after every ten frames. Future work should evaluate a monitored bounded queue, multi node failure behavior, open loop load generation, and longer runs with fixed database versions before making deployment or database selection claims.

ACKNOWLEDGMENT

The authors thank the School of Computer Science and Engineering, Vellore Institute of Technology, for its institutional support and the faculty members who provided guidance during this work.

REFERENCES

[1] C. Lugaresi et al., "MediaPipe: A framework for building perception pipelines," arXiv:1906.08172, 2019.
[2] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, "BlazePose: On-device real-time body pose tracking," arXiv:2006.10204, 2020.
[3] C. Clemente, G. Chambel, D. C. F. Silva, A. M. Montes, J. F. Pinto, and H. P. da Silva, "Feasibility of 3D body tracking from monocular 2D video feeds in musculoskeletal telerehabilitation," Sensors, vol. 24, no. 1, art. 206, 2024, doi: 10.3390/s24010206.

[4] C. M. Powers, “The influence of abnormal hip mechanics on knee injury: A biomechanical perspective,” J. Orthop. Sports Phys. Ther., vol. 40, no. 2, pp. 42–51, 2010, doi: 10.2519/jospt.2010.3337.
[5] R. F. Escamilla, “Knee biomechanics of the dynamic squat exercise,” Med. Sci. Sports Exerc., vol. 33, no. 1, pp. 127–141, 2001, doi: 10.1097/00005768-200101000-00020.
[6] C. Praschl, S. Pritz, O. Krauss, and M. Harrer, “A comparison of relational, NoSQL and NewSQL database management systems for the persistence of time series data,” in Proc. ICECCME, 2022, doi: 10.1109/ICECCME55909.2022.9988333.
[7] S. Pritz, M. Zeinzinger, C. Praschl, O. Krauss, and M. Harrer, “Performance impact of parallel access of time series in the context of relational, NoSQL and NewSQL database management systems,” in Proc. ICECCME, 2023, doi: 10.1109/ICECCME57830.2023.10253446.
[8] J. Mostafa, S. Wehbi, S. Chilingaryan, and A. Kopmann, “SciTS: A benchmark for time-series databases in scientific experiments and industrial Internet of Things,” in Proc. SSDBM, 2022, doi: 10.1145/3538712.3538723.
[9] A. Khelifati, M. Khayati, A. Dignös, D. Difallah, and P. Cudré-Mauroux, “TSM-Bench: Benchmarking time series database systems for monitoring applications,” Proc. VLDB Endow., vol. 16, no. 11, pp. 3363–3376, 2023, doi: 10.14778/3611479.3611532.
[10] A. Papadopoulos et al., “Methodological principles for reproducible performance evaluation in cloud computing,” IEEE Trans. Softw. Eng., vol. 47, no. 8, pp. 1528–1543, 2021, doi: 10.1109/TSE.2019.2927908.
[11] A. Chebotko, A. Kashlev, and S. Lu, “A big data modeling methodology for Apache Cassandra,” in Proc. IEEE BigData Congress, 2015, pp. 238–245, doi: 10.1109/BigDataCongress.2015.41.
[12] A. Lakshman and P. Malik, “Cassandra: A decentralized structured storage system,” ACM SIGOPS Oper. Syst. Rev., vol. 44, no. 2, pp. 35–40, 2010, doi: 10.1145/1773912.1773922.
[13] S. Tseng, B. Nicolae, F. Cappello, and A. Chandramowlishwaran, “Demystifying asynchronous I/O interference in HPC applications,” Int. J. High Perform. Comput. Appl., vol. 35, no. 4, pp. 391–412, 2021, doi: 10.1177/10943420211016511.
[14] S. Salvador and P. Chan, “FastDTW: Toward accurate dynamic time warping in linear time and space,” Intell. Data Anal., vol. 11, no. 5, pp. 561–580, 2007.
[15] Timescale, “Hypertables,” TimescaleDB documentation. [Online]. Available: https://docs.timescale.com/use-timescale/latest/hypertables/. Accessed: Sep. 13, 2026.
[16] B. Lu and X. Yang, “Research on Cassandra data compaction strategies for time-series data,” J. Comput., vol. 11, no. 6, pp. 504–512, 2016, doi: 10.17706/JCP.11.6.504-512.
[17] Apache Cassandra, “Time Window CompactionStrategy,” Apache Cassandra documentation. [Online]. Available: https://cassandra.apache.org/doc/3.11/cassandra/operating/compaction/twcs.html. Accessed: Sep. 13, 2026.